

Five Terms You Need to Know About Agentic AI

Frontier AI agents are pretty capable. They're really good at planning out tasks and writing code with minimal human involvement, but there are a handful of specific pieces under the hood that enable this.

So let's cover five of those pieces—the five terms you need to know about agentic AI.

Let's start with the stuff that's inside the agent and shapes how it behaves.

Inside an agent, of course, there is a model—a large language model. That's what's doing the actual text generation and reasoning. By itself, though, it's just a conversational partner.

What turns it into an agent is the instruction layer wrapped around the model.

1. AGENTS.md

That brings us to term number one: **AGENTS.md**.

So, what is that?

Well, `.md` means Markdown, so it's just a text file. It sits at the root of a project, and whenever the agent starts work in that project, it reads whatever is in that `AGENTS.md` file.

The file tells the agent things like:

- Which commands to run for tests
- Which coding conventions the codebase uses
- Specific setup commands
- Code style rules
- How a pull request title should be formatted

You can really think of it as a README file specifically written for agents.

The agent executes the commands it finds in `AGENTS.md` when they are contextually relevant. So, if the file says, "Run `pnpm test` before committing," then the agent will run `pnpm test` before it makes a commit.

`AGENTS.md` files can also be nested, meaning there can be multiple files.

For example, you might have one at the root of the project and then additional ones inside subprojects, each with its own set of rules. Files closer to the working directory override earlier ones because they appear later in the hierarchy.

`AGENTS.md` was introduced by OpenAI and later contributed to the Agentic AI Foundation under the Linux Foundation.

One wrinkle worth mentioning is that some agents use a different filename.

Claude, for example, uses `CLAUDE.md`—because of course it does.

It's a different name, but more or less the same idea.

So, `AGENTS.md` is read by an agent whenever it starts work in a given project.

But what about knowledge the agent only needs sometimes and that isn't necessarily project-specific?

Let's say the agent needs to know how to build a PowerPoint deck. Loading all of that context every single time the agent starts would clog up the context window for no real reason when the task has nothing to do with PowerPoint slides.

That brings us to term number two.

2. Agent Skills

An **agent skill** is a folder, and inside that folder is a file called `SKILL.md`.

Again, `.md` means Markdown.

The folder can also contain whatever scripts or resources the task needs.

Inside `SKILL.md` is metadata, including a description. That description tells the agent something like:

Invoke me when the user wants to do X.

For example, X could be:

When the user wants to make a PowerPoint presentation.

If the user's request matches that description, the agent pulls the skill in.

If it doesn't match, the skill simply sits there out of the way without taking up context.

Agent skills are another open standard, and they're supported by multiple agent platforms.

So:

`AGENTS.md` **tells an agent how a specific project works.**

An agent skill tells the agent how to perform a specific kind of task.

That's two of our five terms.

The agent now knows what to do, but doing things also means reaching outside the box—outside the AI agent itself.

That's where we're going next.

3. MCP — Model Context Protocol

Agents need to reach all kinds of external systems:

- APIs
- Databases
- Developer tools
- SaaS platforms
- And much more

The challenge is that every one of those targets might have its own interface.

Without some kind of standard, every AI agent would need a custom connector for every external system it might interact with.

That would be a mess.

This brings us to term number three:

MCP — Model Context Protocol.

MCP is an open protocol for connecting AI applications to tools, data sources, and workflows.

It includes something called an **MCP server**.

An MCP server wraps a tool or data source in a standard interface. Any agent that can speak MCP can then communicate with that tool.

Let's say an agent needs to pull data from Notion, or perhaps generate a Stripe payment link.

Whatever the backend system is, the agent speaks MCP to the server, and the server handles the underlying API for that system.

MCP started at Anthropic and is now governed under the Agentic AI Foundation, again under the Linux Foundation.

It has broad industry support.

So that covers agents talking to tools and data.

But what about agents talking to other agents?

That brings us to term number four.

4. A2A — Agent-to-Agent

A2A stands for **Agent-to-Agent**.

It is an open protocol for communication between agents.

Let's imagine a scenario.

We have a **procurement agent** that handles vendor contracts.

Then we have a **finance agent** that approves spending.

I know—financial processing. Try to contain your excitement.

The procurement agent needs to negotiate a contract and then hand it off to finance for approval.

Without A2A, those two agents would need some form of custom integration, or they simply wouldn't coordinate very well.

With A2A, each agent publishes something called an **Agent Card**.

An Agent Card is basically a description of:

- What the agent does
- What capabilities it provides
- How other agents can communicate with it

Other agents can read that card and figure out how to delegate work.

In this example, the procurement agent finds and reads the finance agent's Agent Card and then hands off the contract for approval.

That is A2A.

The A2A standard originated at Google and is now also an open standard under the Linux Foundation.

So:

MCP is how agents talk to tools and data.

A2A is how agents talk to each other.

Now the agent knows what to do, and it knows how to reach outside its own boundaries.

What else?

Sometimes, one agent simply isn't enough.

5. Subagents

Maybe the task is too big for one context window.

For example, imagine an agent reviewing a codebase containing thousands of files. Loading every file into one context window would overwhelm it.

Or maybe the work is **embarrassingly parallel**.

Suppose you need to run a check on 20 different functions, and every check is independent.

You could do them one at a time, but that would be slow.

Running all 20 at once could be dramatically faster.

That brings us to term number five:

Subagents.

A subagent is a child agent that the main agent spawns to perform a specific piece of work.

Each subagent runs in its own fresh context window. It does its job and then returns the result when it is finished.

For example, the main agent could spawn a subagent and say:

Go read 500 files and return a summary.

The subagent handles that work and gives the summary back to the main agent.

This keeps the main agent's context window relatively clean.

You could also have many agents working in parallel.

For example, you might have 20 agents simultaneously handling 20 independent checks.

Subagents are a little different from the other four terms because they are a common pattern in modern agent systems, but they do not really have a single formal standard behind them.

Still, the basic concept appears almost identically across many agent systems:

1. A parent agent spawns one or more child agents.
2. Each child agent receives a specific task and relevant context.
3. The child performs the work.
4. The child returns a result.
5. The parent agent continues with its own context intact.

The Five Terms

So there we have five important terms:

AGENTS.md

Defines how an agent should work inside a specific project.

Agent Skills

Teach an agent how to perform a specific kind of task and are loaded only when relevant.

MCP — Model Context Protocol

Allows agents to communicate with tools, APIs, data sources, and workflows.

A2A — Agent-to-Agent

Allows agents to communicate and delegate work to other agents.

Subagents

Allow an agent to divide large or parallel workloads among multiple child agents with separate context windows.

`AGENTS.md` and agent skills live inside the agent's operating environment and shape how it behaves.

MCP and A2A allow the agent to reach outward—to tools, data, and other agents.

Subagents allow the agent to handle work that is too large, specialized, or parallel to fit comfortably within a single agent context.

That's what a frontier AI agent actually looks like under the hood today.